

BRK (00)		Cycles: 8	Size: 2*
Implied			
Cycle	R/W	Desc.	
-X.2	Read PC	** Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC	Store as Operand.	
2.1		Inc. PC .	
2.2	*** Write to (S +0)	Push PB onto stack.	
3.1			
3.2	*** Write to (S -1)	Push PC .H onto stack.	
4.1			
4.2	*** Write to (S -2)	Push PC .L onto stack.	
5.1		Use Interrupt flags to decide which vector to use. RESET = \$FFFC. NMI = \$FFEA. IRQ = \$FFEE. BRK = \$FFE6.	
5.2	*** Write to (S -3)	Push P onto stack with B flag if software BRK.	
6.1		Dec. S by 4.	
6.2	Read Vector	Store to Address.L.	
7.1		Set I , unset D .	
7.2	Read Vector + 1	Store to Address.H.	
8.1		Enable writes to PC (disabled by NMI/IRQ). Copy Address to PC . Set PB to 0.	
8.2	Read PC	Store as OpCode.	
+X.1			

* Concerning the BRK instruction, you should also note that although its second byte is basically a “don’t care” byte – that is, it can have any value - the BRK (and COP instruction as well) is a two-byte instruction, the second byte sometimes is used as a signature byte to determine the nature of the BRK being executed. When an RTI instruction is executed, control always returns to the second byte past the BRK opcode. — assembly-programming-manual-for-w65c816.pdf

** If NMI, IRQ, or RESET, OpCode is forced to 0 and writes to PC are disabled.

*** If handling a RESET, the writes turn into reads. The databus is populated but ignored.

ORA (01)		Cycles: 6	Size: 2
Direct Indexed Indirect (d,x)			
Cycle	R/W	Desc.	
-X.2	Read PC	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC .	Store as Pointer.	
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address = Pointer.	
2.2		If D.L is not 0, skip the next cycle.	
3.1		Address.H = Pointer.H (fixes high byte of address).	
3.2			
4.1			
4.2	Read Address	Store as Pointer.	
5.1			
5.2	Read Address + 1	Store as Pointer.H.	
6.1			
6.2	Read Pointer: DB	Store as Operand. If M flag is set, skip the next cycle.	
7.1			
7.2	Read Pointer + 1: DB	Store as Operand.H.	
8.1		Perform A = A Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
8.2	Read PC	Store as OpCode.	
+X.1			

COP (02)	Cycles: 8	Size: 2*
Implied		
Cycle	R/W	Desc.
-X.2	Read PC	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC	Store as Operand.
2.1		Inc. PC .
2.2	** Write to (S+0)	Push PB onto stack.
3.1		Inc. PC .
3.2	** Write to (S-1)	Push PC.H onto stack.
4.1		
4.2	** Write to (S-2)	Push PC.L onto stack.
5.1		Set Vector to \$FFE4.
5.2	** Write to (S-3)	Push P onto stack.
6.1		Dec. S by 4.
6.2	Read Vector	Store to Address.L.
7.1		Set I , unset D .
7.2	Read Vector + 1	Store to Address.H.
8.1		Enable writes to PC (disabled by NMI/IRQ). Copy Address to PC . Set PB to 0.
8.2	Read PC	Store as OpCode.
+X.1		

* Concerning the BRK instruction, you should also note that although its second byte is basically a “don’t care” byte – that is, it can have any value - the BRK (and COP instruction as well) is a two-byte instruction, the second byte sometimes is used as a signature byte to determine the nature of the BRK being executed. When an RTI instruction is executed, control always returns to the second byte past the BRK opcode. — assembly-programming-manual-for-w65c816.pdf

** If handling a RESET, the writes turn into reads. The databus is populated but ignored.

ORA (03)		Cycles: 4	Size: 2
Stack Relative (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC .	Store as Pointer.	
2.1		Inc. PC . Set Address to Pointer + S .	
2.2			
3.1			
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1	Store as Operand.H.	
5.1		Perform A = A Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
5.2	Read PC	Store as OpCode.	
+X.1			

TSB (04)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC	Store as Operand. If D.L is not 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1	Store as Operand.H.
5.1		If M flag is set, Z based off (A.L & Operand.L), otherwise set Z based off (A & Operand). Perform Operand = A .
5.2		If M is set, skip the next cycle.
6.1		
6.2	Write to Address + 1	Write Operand.H.
7.1		
7.2	Write to Address	Write Operand.L.
8.1		
8.2	Read PC	Store as OpCode.
+X.1		

ORA (05)		Cycles: 3	Size: 2
Direct (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC	Store as Operand. If D.L is not 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1	Store as Operand.H.	
5.1		Perform A = A Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
5.2	Read PC	Store as OpCode.	
+X.1			

ASL (06)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC	Store as Operand. If D.L is not 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1	Store as Operand.H.
5.1		Perform Operand = Operand << 1. If M flag is set, set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set N based off (Operand.H & \$80) and Z based off Operand.
5.2		If M is set, skip the next cycle.
6.1		
6.2	Write to Address + 1	Write Operand.H.
7.1		
7.2	Write to Address	Write Operand.L.
8.1		
8.2	Read PC	Store as OpCode.
+X.1		

ORA (07)	Cycles: 6	Size: 2
Direct Indirect Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC	Store as Operand. If D.L is not 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
7.1		
7.2	* Read Pointer + 1:Bank	Store as Operand.H.
8.1		Perform A = A Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .
8.2	Read PC	Store as OpCode.
+X.1		

PHP (08)		Cycles: 3	Size: 1
Implied			
Cycle	R/W		Desc.
-X.2	Read PC		Store as OpCode.
1.1			Inc. PC .
1.2			
2.1			Sets Operand.L to P .
2.2	Write to (S +0)		Push Operand.L onto stack.
3.1			
3.2	Read PC		Store as OpCode.
+X.1			